

UNIVERSITY TUNKU ABDUL RAHMAN
FACULTY OF INFORMATION & COMMUNICATION TECHNOLOGY
UCCD3084 GRAPHICS PROGRAMMING FOR EXTENDED REALITY (XR)

FACTORY VR TRAINING SIMULATION
REPAIR, OPTIMIZATION & CUSTOM FEATURE

DESIGN REPORT

PROJECT FEATURE: MAGNET SPEED CONTROL

GROUP 28

WU KAI WEN	2204880
BRIAN LEE ZHEN HUI	2206413
NG CHE TE	2206349
BOO WEI LUN	2202164

SUBMISSION DATE: 29 AUGUST 2026

Table of Contents

Table of Contents.....	2
1. Introduction and Project Scope.....	3
1.1 Project Objective.....	3
1.2 Scope Summary	3
2. Part A – Repair, Lighting and Optimization.....	4
2.1 Onboarding Fade Screen.....	4
2.2 Console Labels Toggle Logic	5
2.3 Factory Floor Lighting.....	6
2.4 Box Pile Lighting.....	7
2.5 Console Lighting.....	9
2.6 Magnet State Visual Feedback.....	10
2.7 Instruction Lock	11
3. Part B – Magnet Speed Control	13
3.1 Feature Overview and Rationale.....	13
3.2 User Interaction.....	13
3.3 Technical Implementation	14
3.4 Speed Mapping and Safety Logic	14
3.5 Feature Requirements Coverage	15
4. Visual Diagrams.....	16
5. Overall VR Experience Requirements.....	17
6. AI Assistance Declaration.....	18
7. Team Contribution	19
8. Conclusion	20

1. Introduction and Project Scope

This project takes the provided factory VR training prototype and turns it into a more polished, usable training simulation. Part A repairs seven deliberately broken UI, lighting, feedback and interaction issues in the starter scene, while Part B adds a custom feature: Magnet Speed Control. The original factory layout and XR rig were kept intact throughout — the work builds on top of the starter project rather than rebuilding it.

Compared against the original starter project, the final Unity scene now has revised UI event wiring, rebaked lighting data, extra light-probe groups, magnet state feedback, updated ray-interaction behaviour, and the new throttle-based speed control system. The custom feature itself is built from an XR slider and a MagnetSpeedControl script, which maps the slider's 0–1 value to a lift speed and adds safety feedback whenever a box is carried above the rated speed.

1.1 Project Objective

- Repair the seven specified defects in the starter scene without replacing the base environment.
- Improve baked-lighting consistency while maintaining a VR-friendly optimization approach.
- Provide clear state feedback for the magnet through colour and particle effects.
- Control access to console interactions during the onboarding sequence.
- Implement an intuitive Magnet Speed Control using a physical XR slider and contextual UI feedback.
- Preserve comfort and usability through teleportation, snap turning, vignette settings, spatial audio and reset behaviour.

1.2 Scope Summary

Area	Implemented Scope
------	-------------------

Part A	Fade screen, console-label toggle, floor lighting, box-pile lighting, console lighting, magnet state feedback, instruction lock.
Part B	Magnet Speed Control with XR slider, 1.0–10.0 m/s mapping, speed readout, safe/warning state and overspeed trip logic.
VR experience	Teleportation, comfort controls, spatial audio support and reset functionality in the final training scene.

2. Part A – Repair, Lighting and Optimization

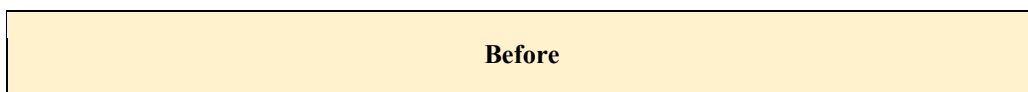
Part A involved finding each deliberately broken behaviour in the starter project and fixing it in the final scene. Each subsection below covers the problem, how it was fixed, and what the user should now experience.

2.1 Onboarding Fade Screen

Problem: In the starter project, the black onboarding fade canvas didn't cover the user's full field of view — it showed up as a misplaced panel floating in the environment instead of a proper full-screen transition.

Implementation: We corrected the fade canvas configuration so it renders as a screen-space overlay. In the serialized scene, `Canvas_Fade` changes from render mode 1 (Screen Space – Camera) in the starter project to render mode 0 (Screen Space – Overlay) in the completed project. This makes the fade independent of world-space placement, so it covers the display consistently no matter where the camera is.

Result: The onboarding fade now behaves as a true full-screen transition and prevents the user from seeing the factory through the edges of the UI.



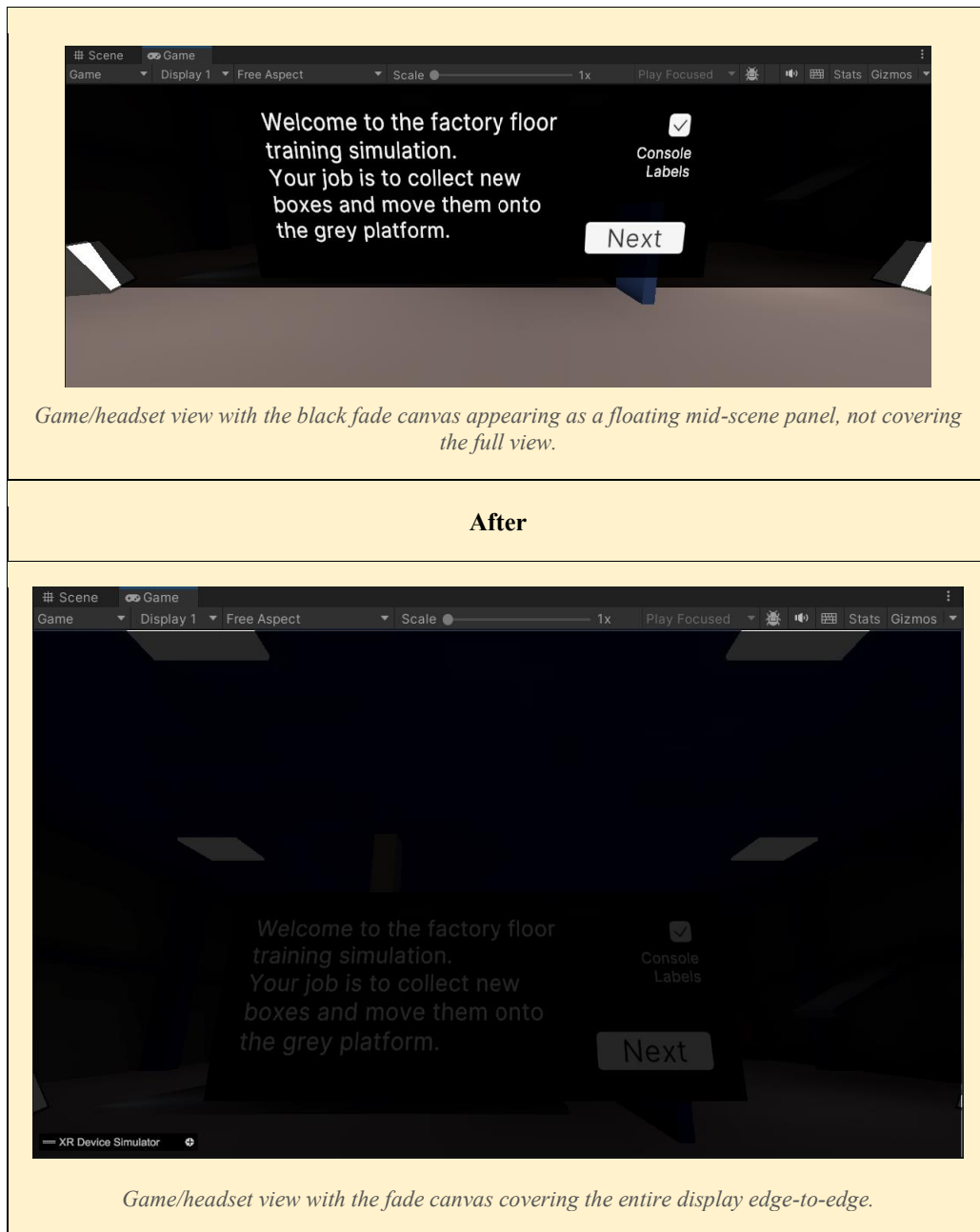


Figure 1: Onboarding Fade Screen

2.2 Console Labels Toggle Logic

Problem: The Console Labels toggle originally called `SetActive` on a `GameObject` reference that caused the whole console-related object to disappear instead of hiding only the text labels.

Implementation: The final scene rewires the toggle event to the `ToggleInterface` component on `Canvas_Labels`. The label canvas is therefore toggled independently while the physical console and controls remain active and visible.

Result: Users can hide or show the explanatory labels without losing access to the console itself.



Before: console visible but labels not shown.



Figure 2: Console Labels Toggle Logic — after fix, all console labels (Magnet On/Off, LIFT readout, Move Vertical/Lateral, Magnet Rotate, Console Up/Down, Clear/Dump Boxes) display correctly together while the console remains fully interactive.

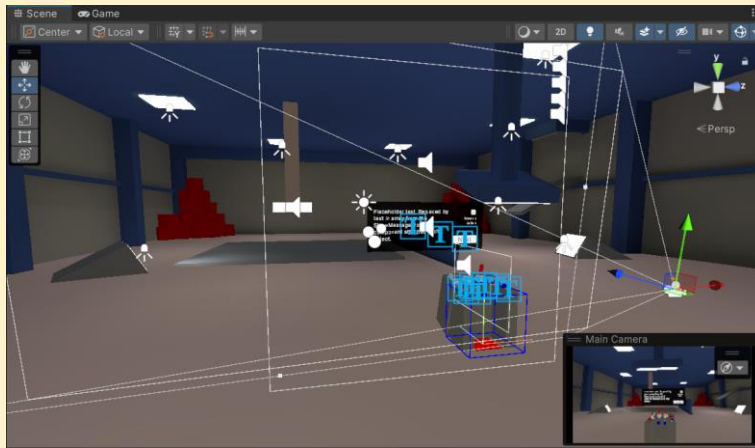
2.3 Factory Floor Lighting

Problem: The factory floor appeared flatter and lower in lighting detail than nearby surfaces in the starter project.

Implementation: The baked-lighting setup was revised, and the lighting data was rebaked. The completed project contains regenerated lighting data and lightmap textures associated with the training scene. The floor material is also updated in the final project.

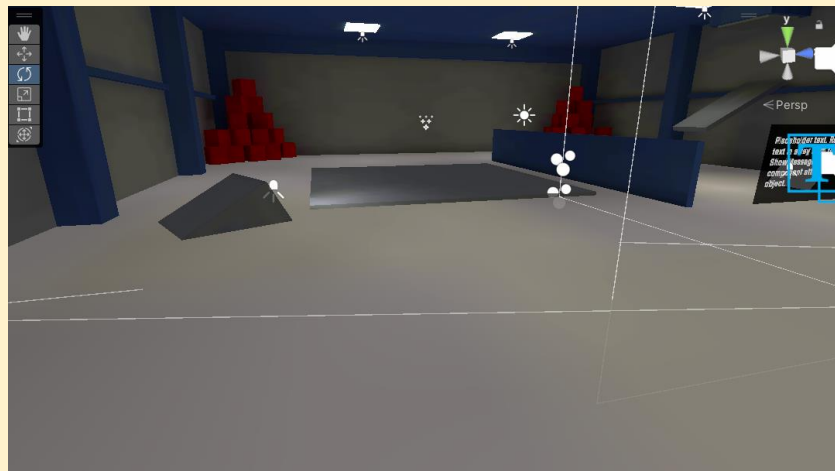
Result: The floor receives more consistent baked illumination and visually matches the surrounding factory surfaces more closely.

Before



Editor or headset view showing the floor noticeably flatter/less detailed than adjacent surfaces.

After



Same viewpoint after rebaking, showing matching lighting detail across the floor and adjacent surfaces.

Figure 3: Factory Floor Lighting

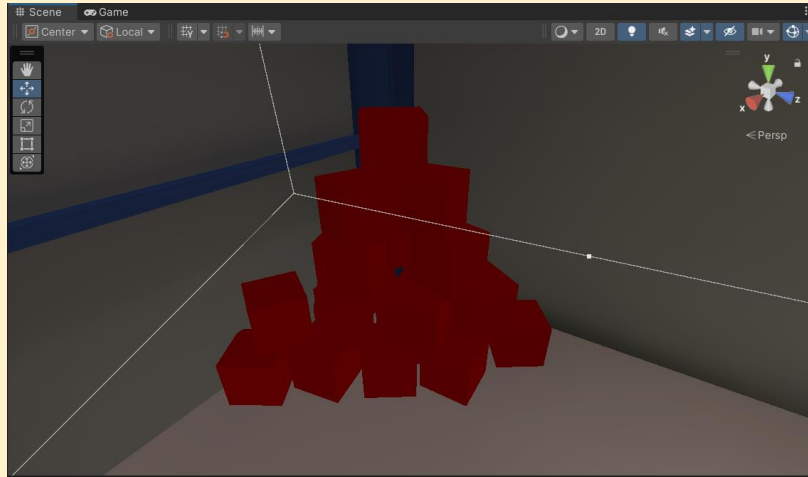
2.4 Box Pile Lighting

Problem: The back-left pile of boxes did not match the baked shadows and lighting of the other box piles.

Implementation: The box geometry was included consistently in the baked-lighting workflow, and the training scene was rebaked. The final project contains updated LightingData and lightmap assets, providing consistent baked shadow information across the static environment.

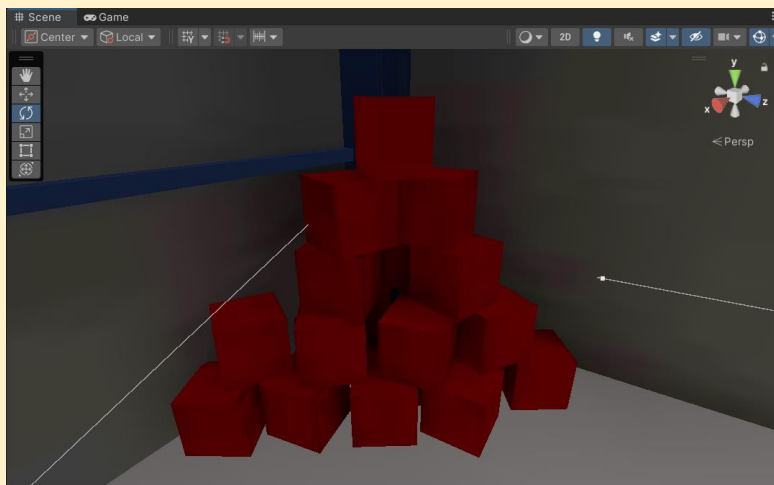
Result: All box piles present a more consistent baked-lighting appearance instead of one pile looking disconnected from the scene lighting.

Before



The back-left box pile with visibly inconsistent/flat shadows compared to other piles.

After



The same pile after rebaking, with shadow detail consistent with the rest of the scene.

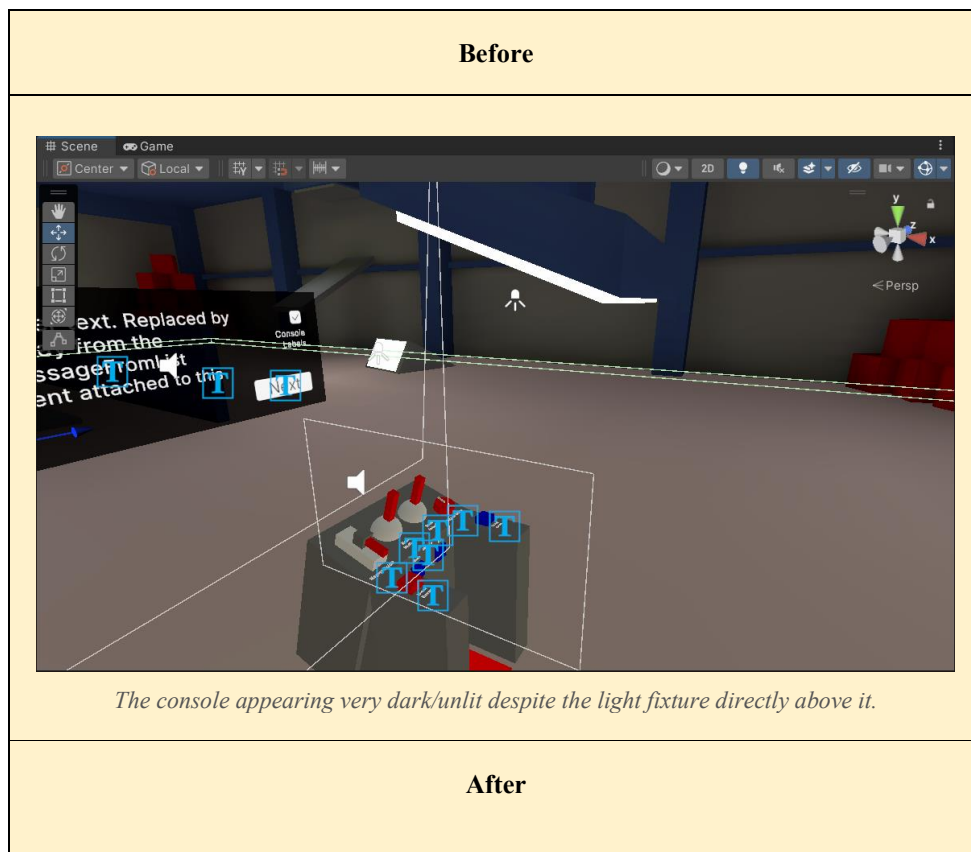
Figure 4: Box Pile Lighting

2.5 Console Lighting

Problem: The console was unusually dark because the baked light above it could not provide appropriate real-time lighting information to the dynamic console components.

Implementation: Light probes were introduced around the relevant scene areas. The completed scene contains additional Light Probe Group objects, allowing non-lightmapped or dynamic objects to sample the baked lighting environment while the overhead light remains baked for optimization.

Result: The console appears illuminated by its surrounding baked environment without converting the overhead light into an expensive real-time light.



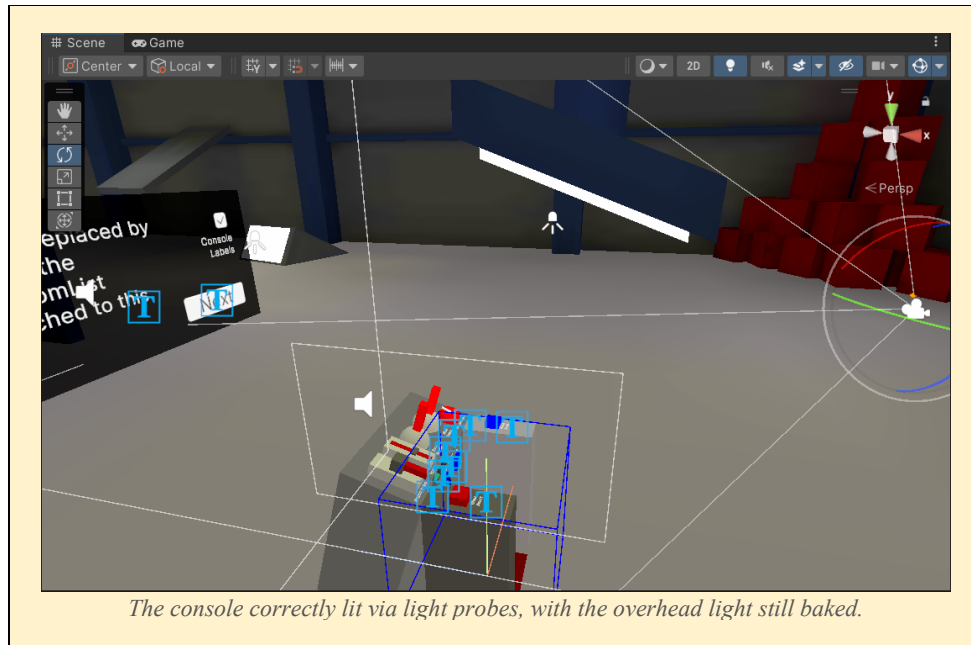


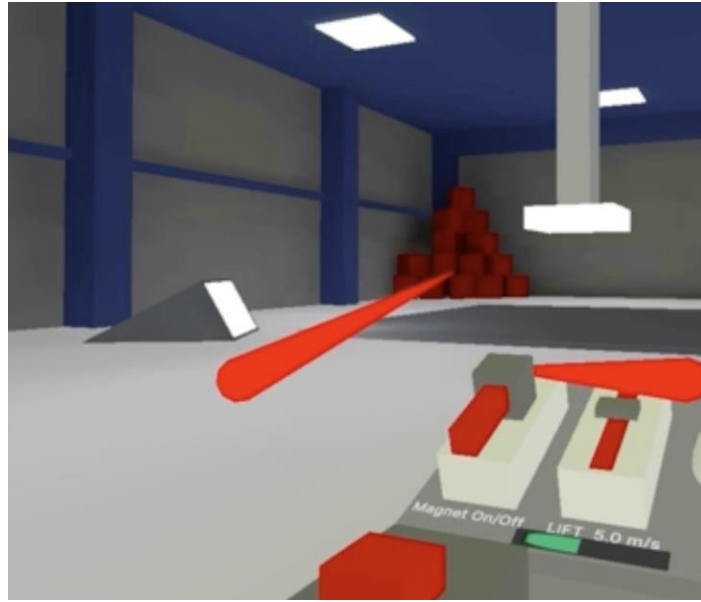
Figure 5: Console Lighting

2.6 Magnet State Visual Feedback

Problem: The original magnet did not clearly communicate whether it was active or inactive.

Implementation: The `SetMaterialColor` script was extended with `activeColor` and `inactiveColor` properties plus `SetActiveColor()` and `SetInactiveColor()` methods. A `Particles_Magnet` particle system was also added as a child of the magnet. These elements are connected to the magnet interaction events so the magnet can provide both a colour-state change and particle feedback while operating.

Result: The user can identify the magnet state immediately, and active lifting is reinforced with a visible particle effect.



Before: magnet with no distinct colour state.

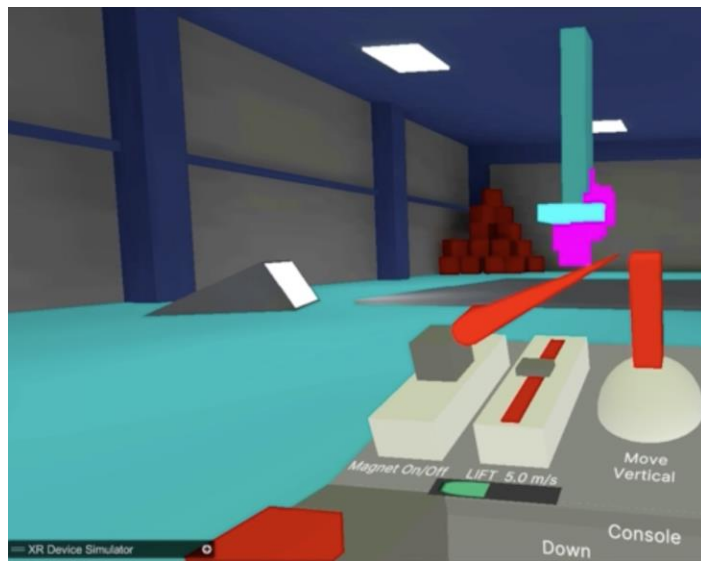


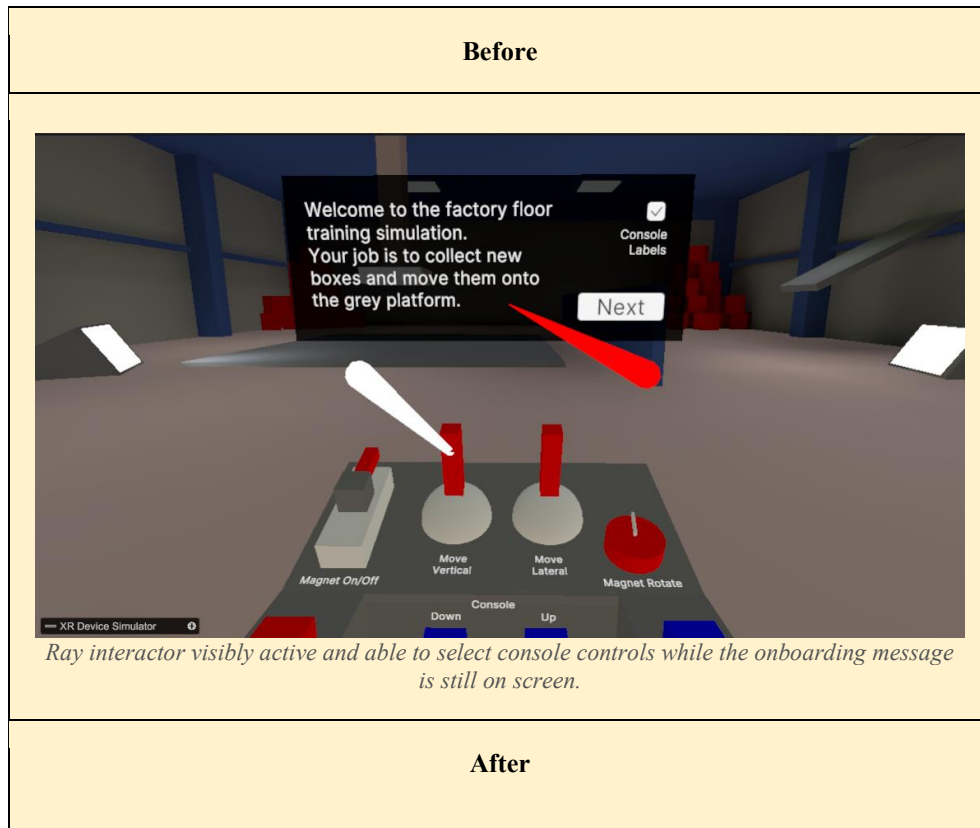
Figure 6: Magnet State Visual Feedback — after fix, the magnet switches to a distinct colour (magenta) while active.

2.7 Instruction Lock

Problem: The starter scene allowed interaction with the console while onboarding instructions were still being read. Ray/direct interaction also remained available when it should have been controlled by the onboarding flow.

Implementation: The final scene revises the ray-interaction setup and event behaviour so console access is restricted during instruction presentation, and the ray interactors are re-enabled only after the onboarding sequence completes. The LeftRay and RightRay objects contain the project ToggleRay behaviour, which is invoked once all onboarding messages have been shown.

Result: The user completes the onboarding sequence without accidentally operating the training console, resulting in a clearer and safer training flow.



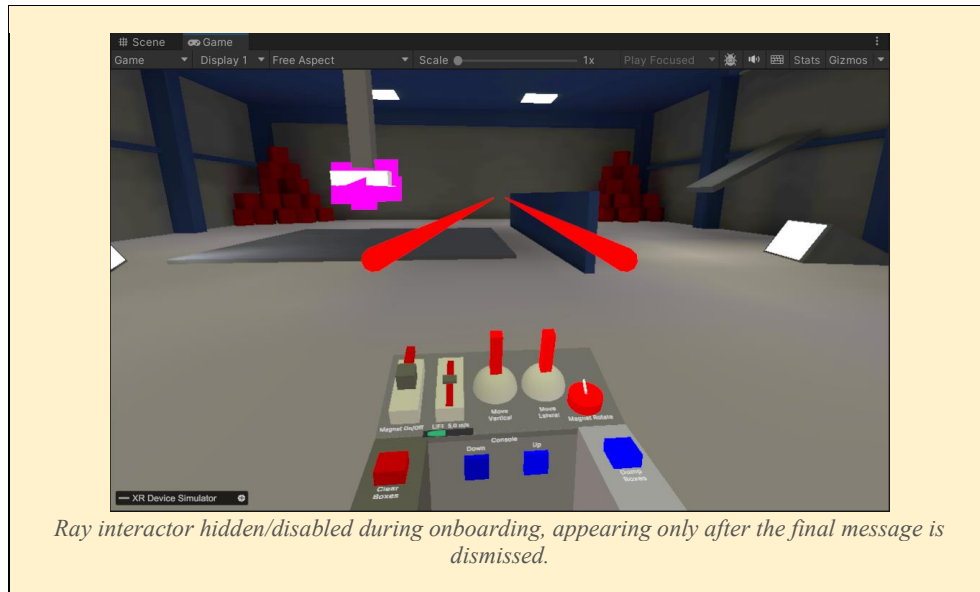


Figure 7: Instruction Lock

3. Part B – Magnet Speed Control

3.1 Feature Overview and Rationale

The additional feature we chose is Magnet Speed Control. In the starter environment the magnet only moves at one fixed speed, so the completed system adds a physical throttle-style XR slider to the console that lets the trainee change the lifting speed before or during operation. We picked this feature because it adds meaningful control to an existing factory mechanism instead of bolting on an unrelated mini-game, and it fits the training goal well: a trainee can work faster, but running too fast while carrying a load triggers a warning and can trip the magnet.

3.2 User Interaction

- The trainee approaches the console and grabs/interacts with the throttle slider.
- The XR slider reports a normalized value between 0 and 1.
- MagnetSpeedControl maps the value linearly to a lift speed between 1.0 m/s and 10.0 m/s.
- The selected speed is written directly to the existing MoveWithVelocity component that drives magnet movement.
- A TextMeshPro readout displays the current speed in the form "LIFT x.x m/s".

- The bar fill follows the throttle position and changes to a warning colour when the selected speed exceeds the rated speed.
- When a box is in the magnet socket and the speed remains above the 6.0 m/s rated limit for 2 seconds, warning/trip UnityEvents trigger safety feedback.
- The reset function clears the overspeed timer and returns the throttle to its configured default value.

3.3 Technical Implementation

Component	Role in the Feature
XRSlider (Throttle)	Physical VR control used by the trainee to set a 0–1 throttle value.
MagnetSpeedControl.cs	Maps slider value to speed, updates UI, checks load/overspeed state and raises UnityEvents.
MoveWithVelocity	Existing mover whose speed field is updated by MagnetSpeedControl.
XRSocketInteractor	Detects whether the magnet is currently carrying a selected box.
TextMeshProUGUI readout	Displays the current lift speed numerically.
UI Image bar	Displays normalized throttle position and switches between safe/warning colours.
UnityEvents	OnWarning, OnTrip and OnSafe decouple the speed logic from audio, particle and other feedback.

3.4 Speed Mapping and Safety Logic

The feature maps the XR slider input to a usable physical speed with `Mathf.Lerp(minSpeed, maxSpeed, Mathf.Clamp01(value))`. With the current settings, that's a minimum of 1.0 m/s and a maximum of 10.0 m/s, with the slider defaulting to position 0.44. The load-safety check uses a rated speed of 6.0 m/s, and overspeed only counts while the XR socket reports it's actually carrying a box — so the magnet moving empty at high speed never triggers a false warning.

Once a carried load goes above the rating, the script fires `OnWarning` once and starts accumulating overspeed time. If that lasts through the 2.0-second grace period, `OnTrip` fires and the timer resets;

dropping back to a safe speed fires OnSafe instead. This state-based approach works well for training purposes since the trainee can directly see the cause-and-effect between what they did, the system's limits, and the feedback they get.



Slow speed: throttle near default, bar mostly green/safe.

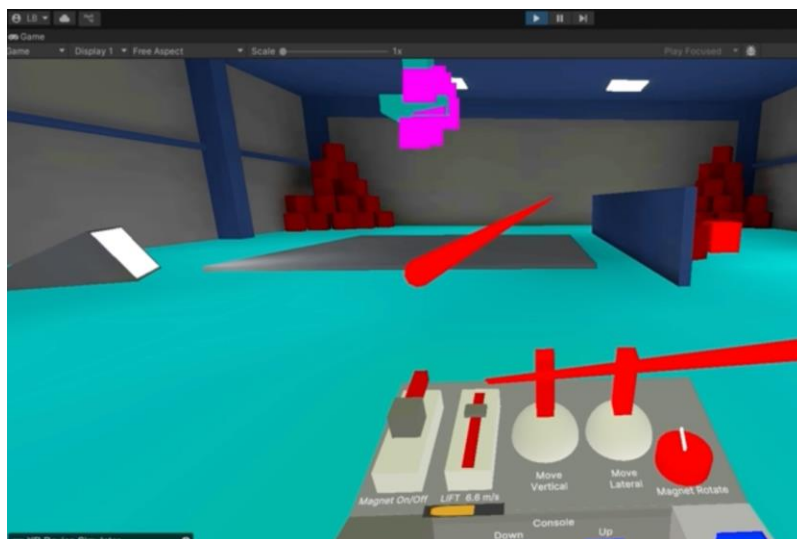


Figure 8: Magnet Speed Control — throttle pushed to a higher setting; readout and bar reflect the increased speed.

3.5 Feature Requirements Coverage

Assignment Requirement	How the Feature Addresses It
Interactable object	The throttle is an XRSlider that the trainee physically grabs and slides in VR, with proper collider and Rigidbody physics.

Correct / incorrect feedback	Safe and overspeed states are distinguished through the readout/bar colour state and UnityEvent-driven warning/trip feedback.
Contextual UI	The speed readout presents the current lift speed and the UI bar represents throttle position/status.
Particle effect	Particle feedback (Particles_Magnet) plays while the magnet is actively lifting, tied to the magnet's active state.
Comfort-friendly design	The feature changes the industrial mechanism speed and UI state without forcing rapid camera motion.

4. Visual Diagrams

The diagrams below cover both the Part B user-action/system-response flow and the relevant Part A interaction architecture, giving a clear visual explanation of how the simulation works.

Magnet Speed Control - User Action and System Response

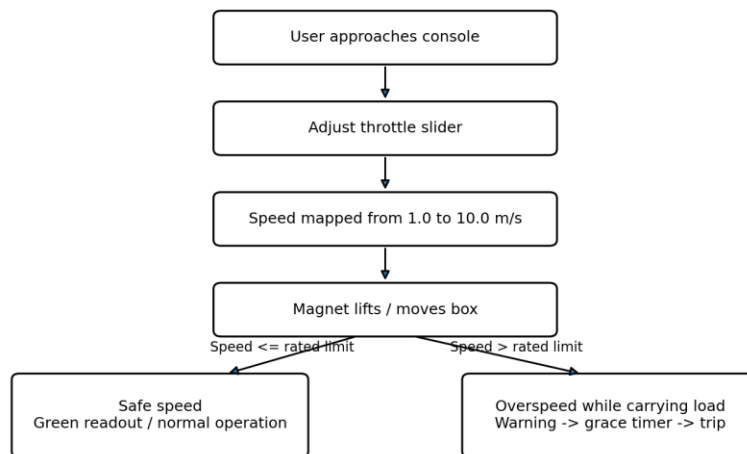


Figure 9: Magnet Speed Control User Flow

Part B Scene / Logic Architecture

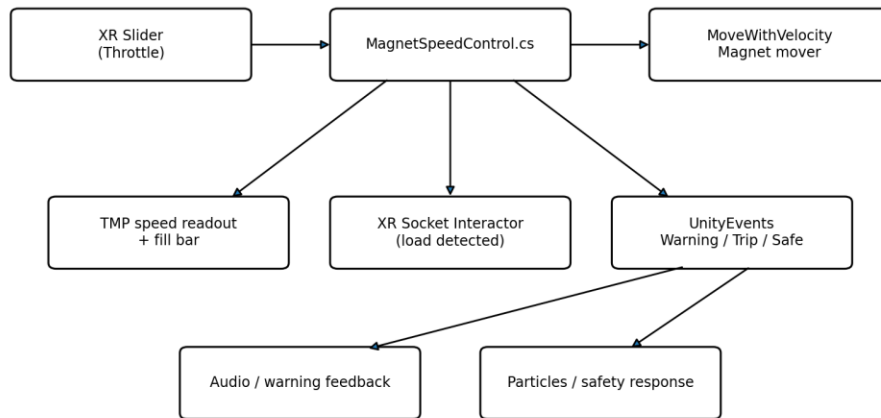


Figure 10: Part B Component and Logic Architecture

Part A Instruction-Lock Logic

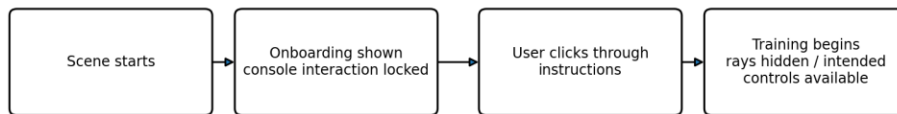


Figure 11: Part A Onboarding / Instruction-Lock Flow

5. Overall VR Experience Requirements

Beyond the repair tasks and the custom feature, the final training scene also needs to meet the required VR usability features. The table below reflects what we've actually confirmed through Editor/simulator testing so far; anything marked "to confirm" still needs a pass on the physical headset before final submission, since things like head/hand tracking, the comfort vignette, and spatialised audio can't be fully validated in the Editor alone.

Requirement	Status	Notes
Teleportation	To confirm on headset	Locomotion is retained from the base XR rig; final confirmation requires a headset pass since simulator testing does not reproduce real locomotion comfort.
Snap turning	To confirm on headset	Snap-turn input action exists on the XR rig; needs a headset pass to confirm behaviour feels correct.
Vignette option	To confirm on headset	Comfort vignette setting must be verified in the final headset build and shown in the demonstration video.
Spatial audio	To confirm on headset	Scene includes audio-source objects for factory mechanisms; directional/spatial behaviour should be confirmed by turning the head during playback.
Reset	Implemented and tested	Reset Button (XRButton) triggers box reset, audio feedback, and MagnetSpeedControl.ResetSpeed(), confirmed working via Editor testing.

6. AI Assistance Declaration

As required by the assignment's policy on AI-assisted code generation, the AI assistance used during development is declared below and has been checked and verified by the student(s) named before being included in the submitted project.

Area	Nature of AI Assistance	Verified By
XRButton.cs (Reset Button, Move Up/Down Button, Spawn Button)	AI assistance was used to diagnose a <code>NullReferenceException</code> thrown from <code>FindButtonHeight()</code> . The cause was that <code>EndPress()</code> cleared the tracked <code>hoverInteractor</code> whenever any interactor stopped hovering, even if a different interactor was still hovering the same button, leaving <code>isHovered</code> true with a null reference on the next update. A guard was added to <code>EndPress()</code> to only clear the reference when the exiting interactor matches the tracked one, plus a null check in <code>ProcessInteractable()</code> as a	Brian Lee Zhen Hui

	safeguard. The fix was applied manually in the Unity script and tested in-Editor before being kept.	
Scene interaction wiring (Throttle, Reset Button, Move Up/Down Button, Spawn Button, Knob, Lever, Joystick_Raise, Joystick_Move)	AI assistance was used to inspect the Unity scene file and identify that several interactable objects had no Interaction Manager reference and/or an empty Colliders list, which was preventing the XR ray and direct interactors from detecting them. The missing references were then reassigned manually in the Unity Editor Inspector.	Ng Che Te
XR Device Simulator testing workflow	AI assistance was used to identify the correct simulator controls (selecting a controller, toggling Translate/Rotate mode) needed to physically test push-button and slider interactions in Play mode, and to add temporary Debug.Log statements to XRButton.cs to confirm press detection during testing. The debug logging was removed before final submission.	Wu Kai Wen

7. Team Contribution

Name	Student ID	Primary Responsibilities
Wu Kai Wen	2204880	Part B implementation and video recording.
Brian Lee Zhen Hui	2206413	Part A repairs.
Ng Che Te	2206349	Part B implementation and video recording.
Boo Wei Lun	2202164	Report writing.

8. Conclusion

The completed Factory VR Training Simulation improves on the supplied prototype through targeted UI, lighting, interaction and feedback repairs, while keeping the original factory and XR structure intact. The most noticeable Part A improvements are the corrected full-screen onboarding fade, the independent console-label toggle, the rebaked lighting and light-probe support, the magnet's colour/particle feedback, and the locked-down interaction flow during instructions.

Magnet Speed Control extends the existing magnet system with a high-fidelity XR control that's directly relevant to factory training: the trainee can physically adjust the lift speed, watch a numerical and visual readout respond, and get a safety-oriented response if they operate above the rated speed while carrying a load. The implementation reuses the existing MoveWithVelocity and XR socket systems and adds one focused MagnetSpeedControl component, which keeps the feature modular and easy to hook up to audio, particle and reset feedback through Unity Events.